

Colegio tecnico UFMG

RELATÓRIO AS-BUILT
Simulador de Controle Aeroportuário em C

Nome: Rafael Rodrigues

Curso: ADS

periodo: 2

Documentação da arquitetura efetivamente projetada, construída e testada, com foco no uso de fila de prioridade, lista duplamente encadeada, fila FIFO e pilha.

1 Mapeamento de Requisitos

A proposta de trabalho é a construção de um protótipo de controle aeroportuário em linguagem C, utilizando problemas de aproximação e pouso, circulação terrestre e despacho de bagagens como contexto para aplicação prática de estruturas de dados. A solução foi planejada em três componentes independentes, posteriormente integrados por um simulador.

A proposta não busca reproduzir integralmente um sistema real de controle de tráfego aéreo. O objetivo acadêmico é representar as regras essenciais do problema de forma suficientemente clara para testar fila de prioridade, lista duplamente encadeada, fila FIFO e pilha, observando o comportamento dessas estruturas durante um ciclo completo de uma aeronave.

Proposição da solução

A solução foi pensada a partir da separação das responsabilidades. O primeiro módulo decide a ordem de atendimento das aeronaves que aguardam pouso. O segundo representa a infraestrutura terrestre necessária para pista, taxiway e portão. O terceiro representa o manifesto e a restrição física de acesso ao porão. O simulador coordena esses módulos e fornece uma sequência temporal para verificar se as estruturas trabalham corretamente em conjunto.

Divisão dos códigos

Foram entregues sete arquivos de código-fonte, um arquivo para cada uma das 3 etapas mais o cabeçalho e a simulação. Cada componente possui um arquivo .h, responsável pelas definições públicas, estruturas e protótipos, e um arquivo .c, responsável pela implementação.

A integração é realizada por simulador.c. Essa divisão reduz o acoplamento, permite testar cada etapa separadamente e mantém a responsabilidade de cada módulo identificável.

Arquivo	Etapas	Responsabilidade principal
controle_aereo.h	Componente 1	Tipos, estados, Aviao e interface da fila de prioridade.
controle_aereo.c	Componente 1	Heap binário, cálculo de prioridade e geração de aeronaves.
controle_solo.h	Componente 2	Nós da taxiway, pistas, portões e estrutura Aeroporto.
controle_solo.c	Componente 2	Lista duplamente encadeada e operações do controle terrestre.
despacho_porao.h	Componente 3	Itens, manifesto, porão e interface das operações de despacho.
despacho_porao.c	Componente 3	Pilha, double-check e interceptação de bagagem.
simulador.c	Integração	Ciclo temporal, fila de saída, logs e coordenação dos três módulos.

Pré-requisitos e execução

O projeto foi desenvolvido para compilação com GCC em ambiente compatível com C11. A biblioteca matemática é vinculada por `-lm` devido às operações utilizadas na validação de valores numéricos do despacho. Não são necessárias bibliotecas externas de terceiros. Os aeroportos são parametrizados por arquivos de configuração, e os logs são gravados em arquivos de texto durante a simulação conforme instruções do README.md do projeto.

A arquitetura conjunta foi adotada porque os componentes possuem dependências operacionais, mas não devem perder sua separação conceitual. Uma aeronave só pode pousar se as regras do Componente 1 a selecionarem e se o Componente 2 indicar condições de pista e solo. Depois do estacionamento, o Componente 3 é acionado antes de a aeronave seguir para a fila de saída.

2 Componente 1

O Componente 1 implementa a Aproximação e Autorização de Pouso. Seu problema principal é decidir qual aeronave deve ser atendida primeiro quando várias aguardam autorização, respeitando emergências, voos comerciais e voos de carga ou particulares.

2.1 Arquitetura

A estrutura escolhida foi uma fila de prioridade baseada em heap binário armazenado em vetor. Cada posição do vetor guarda um ponteiro para uma estrutura `Aviao`. A raiz do heap representa a aeronave de maior prioridade operacional. Quando ocorre uma inserção ou remoção, apenas o caminho necessário da árvore é reorganizado.

A prioridade é calculada automaticamente. Combustível abaixo de 10% ou emergência médica classificam a aeronave como prioridade 1. Na ausência de emergência, voos comerciais recebem prioridade 2 e voos de carga ou particulares recebem prioridade 3. Empates são resolvidos pela ordem de chegada.

2.1.1 Diagramação

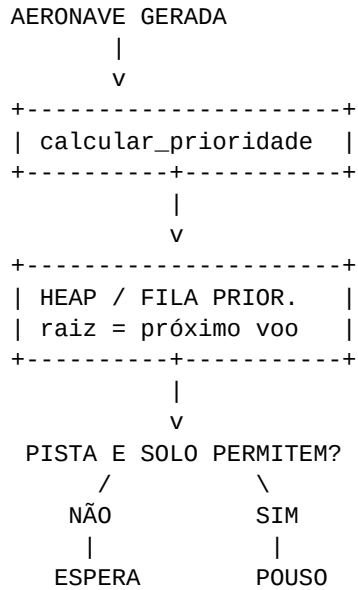
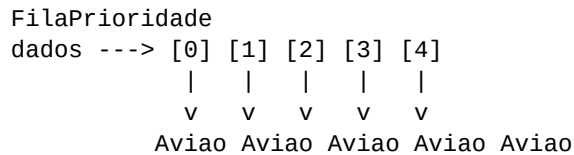
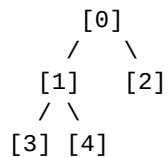


Figura 1 — Fluxo simplificado do Componente 1.



Representação lógica:



```

pai(i) = (i - 1) / 2
esquerda = 2*i + 1
direita  = 2*i + 2
  
```

Figura 2 — Heap binário representado por índices de um vetor.

2.1.2 Detalhamento dos Algoritmos e Estrutura dos Dados

A estrutura Aviao contém identificação, combustível, emergência médica, tipo de voo, prioridade, ordem de chegada, sentido de circulação, estado operacional, posição na taxiway, portão, pista e marcas de tempo. Esses dados permitem que o mesmo objeto acompanhe todo o ciclo do avião.

A FilaPrioridade mantém Aviao **dados, tamanho e capacidade. O vetor armazena ponteiros, evitando cópias completas da estrutura Aviao durante as trocas do heap. A relação entre pai e filhos é obtida matematicamente pelos índices, dispensando ponteiros específicos de árvore.

O benefício principal é operacional: consultar a aeronave prioritária ocorre diretamente na raiz. Inserções e remoções exigem reorganização logarítmica do heap, enquanto uma lista não ordenada precisaria ser percorrida para localizar o melhor candidato. Em memória, o vetor evita a sobrecarga de ponteiros de encadeamento por nó. Uma pilha em vetor também seria compacta, mas sua regra LIFO não representaria as prioridades do pouso.

Como ponto de escalabilidade, a implementação trabalha com capacidade controlada e memória do vetor. Em uma solução de grande escala seria preciso considerar redimensionamento, persistência, concorrência e sincronização entre múltiplos controladores.

2.1.3 Regras de Negócio

A política de pouso não depende apenas da posição na fila. O pensamento foi de tentar planejar uma torre de controle, antes da autorização, o simulador consulta as condições fornecidas pelo controle terrestre. Uma pista livre não basta quando a entrada da taxiway está bloqueada ou quando o solo não possui capacidade operacional para receber a aeronave.

Em uma implementação real o ideal é implementar fatores como separação mínima entre aeronaves, meteorologia como velocidade do vento e chuva, categoria da pista, peso, tipo de aproximação, restrições, fechamento temporário de pista, comunicação com centros de controle e persistência das autorizações. São porpostas para um trabalho com fins reais.

2.2 Componente 2

O Componente 2 implementa a Circulação e Gerenciamento de Solo. A estrutura principal é uma lista duplamente encadeada, na qual cada nó representa uma posição da taxiway. O objetivo é permitir que a aeronave percorra a mesma sequência de posições nos dois sentidos.

2.2.1 Arquitetura

Cada nó possui:

- identificador
- estado de ocupação
- ponteiro para a aeronave presente
- ponteiro para o nó anterior
- ponteiro para o próximo nó.

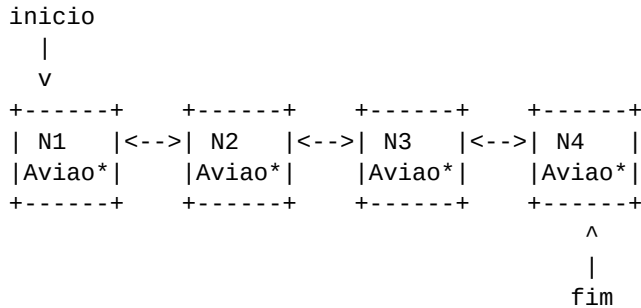
O Aeroporto mantém ponteiros para início e fim da lista, além de vetores para pistas e portões.

O encadeamento duplo foi escolhido porque permite:

N1 -> N2 -> ... -> Nn e o percurso inverso Nn -> ... -> N2 -> N1

sem criar duas estruturas distintas. A política de circulação implementada controla o sentido utilizado, reduzindo conflitos frontais no modelo simplificado.

2.2.2 Diagramação



Sentido A: N1 -> N2 -> N3 -> N4

Sentido B: N4 -> N3 -> N2 -> N1

Figura 3 — Lista duplamente encadeada utilizada para representar as posições da taxiway.

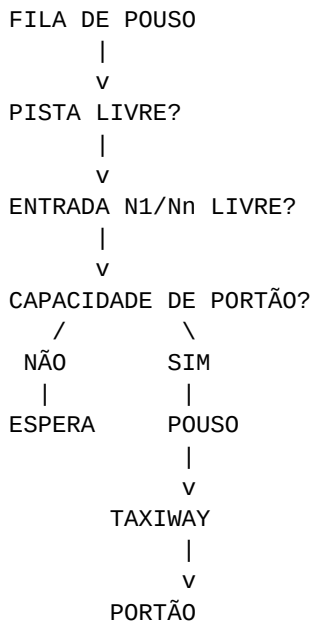


Figura 4 — Condições de solo que participam da autorização de pouso.

2.2.3 Detalhamento dos Algoritmos e Estrutura dos Dados

A estrutura No guarda id, ocupado, Aviao *aviao, No *proximo e No *anterior. Portao guarda id, ocupação e aeronave. Pista guarda id, ocupação, aeronave e instante de liberação. Aeroporto reúne nome, quantidades, início e fim da lista e os vetores de pistas e portões.

A navegação é simples porque cada posição conhece diretamente seus vizinhos. A movimentação só ocorre quando o destino é válido e está livre. No último nó, a aeronave tenta ocupar um portão; se não houver portão disponível, permanece aguardando em vez de desaparecer do fluxo.

A lista encadeada oferece flexibilidade de navegação e evidencia o uso de ponteiros, embora tenha custo adicional de memória pelos campos anterior e próximo. o custo é aceitável, pois o comportamento bidirecional é parte central do experimento.

Em escalas maiores, a lista linear não representa adequadamente cruzamentos, desvios ou caminhos alternativos. O custo de localizar arbitrariamente uma posição também cresce com o percurso da lista. Esses pontos limitam o modelo para aeroportos complexos.

2.2.4 Regras de Negócio

O controle terrestre impede o pouso quando o solo está completamente congestionado. A autorização verifica pista, entrada da taxiway e capacidade de portão. O movimento entre nós impede ocupação simultânea da mesma posição e a circulação em sentidos opostos é controlada para evitar conflitos.

Uma solução real poderia substituir ou complementar a lista por um grafo. Nesse modelo, interseções e posições seriam vértices e os trechos de taxiway seriam arestas, permitindo calcular rotas, bloquear trechos, representar múltiplas pistas e estudar conflitos em topologias mais próximas de um aeroporto real.

2.3 Componente 3

O Componente 3 implementa Despacho de Porão, Cadastro Prévio e Inspeção. Ele mantém uma representação lógica do manifesto e uma representação física do porão. A estrutura principal para o acesso físico é uma pilha em vetor.

2.3.1 Arquitetura

A pilha foi escolhida porque o requisito determina que o último item embarcado seja o primeiro acessível. Essa é exatamente a política LIFO. O manifesto, por sua vez, permanece como um vetor de registros para consulta e comparação com o item retirado fisicamente.

2.3.2 Diagramação

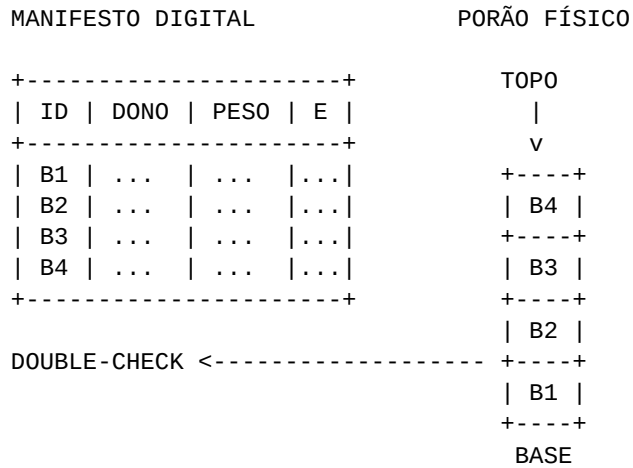


Figura 5 — Separação entre cadastro digital e ordem física LIFO do porão.



B3 retorna primeiro e B4 depois.
A ordem relativa dos itens restantes é preservada.

Figura 6 — Intercepção de emergência e restauração da pilha.

2.3.3 Detalhamento dos Algoritmos e Estrutura dos Dados

ItemManifesto contém:

- identificador
- dono
- peso
- estado.

Manifesto mantém um vetor de itens e sua quantidade.

Porão mantém outro vetor e a quantidade de itens fisicamente embarcados.

O topo da pilha é obtido pela posição quantidade - 1, dispensando ponteiros de encadeamento.

Durante o descarregamento, o item do topo é retirado e comparado ao manifesto. O double-check verifica a existência do identificador e divergências relevantes, como dono, peso e estado. Essa separação entre cadastro e conteúdo físico permite que o sistema detecte inconsistências em tempo de operação.

A pilha possui baixo custo para inserção e retirada no topo e utiliza memória contígua. Para a inspeção de um item profundo, entretanto, é necessário remover temporariamente todos os itens posicionados acima dele. Esse custo não é um defeito da estrutura no contexto do problema: ele representa justamente a restrição mecânica proposta para o porão.

Em uma solução maior, o manifesto em vetor pode exigir busca mais eficiente, persistência em banco de dados, índices por passageiro e identificação, controle transacional e integração com sistemas de segurança.

2.3.4 Regras de Negócio

Antes do embarque, os itens são cadastrados no manifesto.

Durante o descarregamento, cada retirada física é validada. Em uma interceptação de emergência, os itens acima da bagagem alvo são removidos temporariamente, o alvo é isolado e registrado e os demais itens são recolocados de forma a preservar sua ordem original.

Em uma aplicação real poderiam existir etiquetas RFID ou códigos de barras, integração com passageiros e voos, cadeia de custódia, trilha de auditoria persistente, classificação de cargas especiais e comunicação com sistemas de alfândega e segurança. O protótipo utiliza dados simulados para testar a estrutura.

2.4 Integração e Simulador

E finalmanete, simulador.c é responsável por integrar os componentes e testar as estruturas em uma sequência operacional única. Ele gera aeronaves, alimenta o heap, consulta as condições do solo, movimenta aeronaves na lista, aciona o despacho no portão e organiza a saída para decolagem, transforma cada uma das etapas anteriores em um modelo operacional. A principio cada etapa era individualizada com sua própria main, mas temporizar o programa trouxe a possibilidade de olhar os logs e identificar defeitos de funcionamento no fluxo de aviões, por esse motivo a decolagem e o despacho foram inseridos aqui, os aviões todos decolam, não vão para garagem.

2.4.1 Arquitetura e fluxo de teste

A integração foi construída de modo que cada módulo mantenha sua responsabilidade. O simulador não reimplementa o heap, a lista ou a pilha; ele chama as operações públicas de cada componente e controla os estados e tempos necessários para avançar o ciclo. É uma implementação complementar para as 3 etapas, mas por estruturas da linguagem, desenvolve-la requer compilação de todos arquivos juntos.

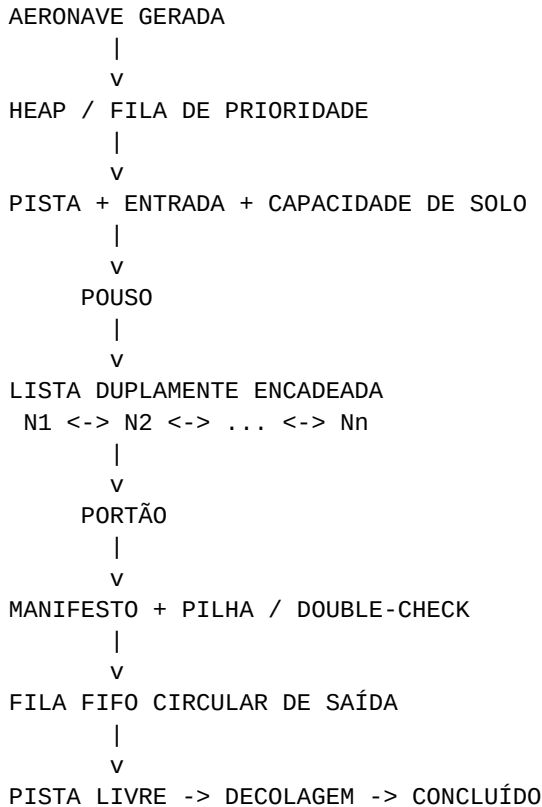


Figura 7 — Fluxo integrado utilizado para exercitar as estruturas do sistema.

2.4.2 Fila de saída, temporização e logs

A saída para decolagem utiliza uma fila FIFO circular em vetor. Diferentemente da aproximação, essa etapa não aplica o mesmo heap de prioridades. A fila circular preserva a ordem de entrada e reutiliza posições do vetor sem deslocar todos os elementos após cada remoção.

O simulador utiliza tempos de operação para pouso, movimentação, permanência em solo e decolagem.

Os logs foram separados por responsabilidade:

torre_controle.log registra eventos de controle aéreo

e terrestre; despacho_porao.log registra manifesto, descarregamento

e double-check; simulador.log mantém o histórico geral.

Nos testes do protótipo, o critério principal de validação foi observar se as aeronaves conseguiam completar o ciclo sem permanecer indevidamente presas nas filas ou estruturas. O simulador também permitiu identificar e corrigir situações como autorização de pouso com solo congestionado, conflitos de movimentação na taxiway e permanência no último nó enquanto um portão não estivesse disponível.

2.4.3 Limitações da integração

Os tempos são simulados, os dados das aeronaves e bagagens são fictícios e a infraestrutura é simplificada.

Não há concorrência real entre processos, integração com sensores ou sistemas externos, nem modelagem completa das regras aeronáuticas. Essas limitações são intencionais: o simulador foi construído para observar e testar estruturas de dados em C, e não para operar como sistema aeroportuário real.